



**ΠΑΝΕΠΙΣΤΗΜΙΟ
ΠΑΤΡΩΝ**
UNIVERSITY OF PATRAS

**ΤΜΗΜΑ ΗΛΕΚΤΡΟΛΟΓΩΝ ΜΗΧΑΝΙΚΩΝ ΚΑΙ
ΤΕΧΝΟΛΟΓΙΑΣ ΥΠΟΛΟΓΙΣΤΩΝ**

Μάθημα: Ρομποτικά Συστήματα ΙΙ

2^η ατομική εργασία

Σάλιαρης Γεώργιος - ΑΜ: 1092584

**Υπεύθυνος καθηγητής: Κωνσταντίνος
Χατζηλυγερούδης**

Ακαδημαϊκό έτος: 2025-26

Άσκηση 1

(Βλ. το τέλος της εργασίας για το derivation των εξισώσεων του συστήματος)

Έχουμε ένα σύστημα *quadrotor* το οποίο περιγράφεται από τις παρακάτω εξισώσεις:

$$\mathbf{x} = \begin{bmatrix} x \\ y \\ \theta \\ \dot{x} \\ \dot{y} \\ \dot{\theta} \end{bmatrix} \in \mathbb{R}^6$$

$$\mathbf{u} = \begin{bmatrix} T \\ \tau \end{bmatrix} \in \mathbb{R}^2$$

$$\dot{\mathbf{x}} = f(\mathbf{x}, \mathbf{u}) = \begin{bmatrix} \dot{x} \\ \dot{y} \\ \dot{\theta} \\ \frac{1}{m} \left(-\sin\theta(T + f_y) + \cos\theta f_x \right) \\ \frac{1}{m} \left(\cos\theta(T + f_y) + \sin\theta f_x \right) - g \\ \frac{\tau - f_\theta}{J} \end{bmatrix} \in \mathbb{R}^6$$

Ενώ έχουμε τους εξής περιορισμούς στα *control inputs*:

$$\frac{1}{2}mg \leq T \leq \frac{3}{2}mg \quad (1.1)$$

$$-0.5 \leq \tau \leq 0.5 \quad (1.2)$$

Τα *drag forces* είναι:

$$f_x = -kv_x|v_x| \quad (1.3)$$

$$f_y = -k v_y |v_y| (1.4)$$

$$f_\theta = -k_\theta \dot{\theta} (1.5)$$

Οι παράμετροι του συστήματος είναι:

$$g = 9.81 \frac{m}{s^2} (1.6)$$

$$m = 1kg (1.7)$$

$$J = 0.05 kgm^2 (1.8)$$

$$k_x = k_y = 0.3 (1.9)$$

$$k_\theta = 0.02 (1.10)$$

Οι παράμετροι της προσομοίωσης:

DT – το χρονικό βήμα της προσομοίωσης

TF – ο συνολικός χρόνος προσομοίωσης

XREF – το επιθυμητό *x* του κέντρου μάζας του quadrotor

YREF – το επιθυμητό *y* του quadrotor

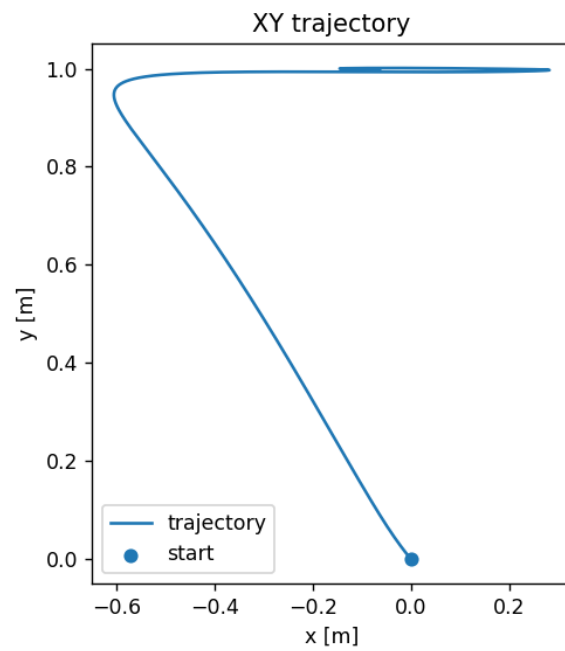
XCTL – παράμετρος η οποία μας επιτρέπει να ελέγχουμε το θ , ειδικά είναι 0.

Εμείς προσομοιώνουμε το σύστημά μας μέσω *RK4* και έχοντας ενσωματώσει *PD* ελεγκτές για το *thrust force* και την ροπή.

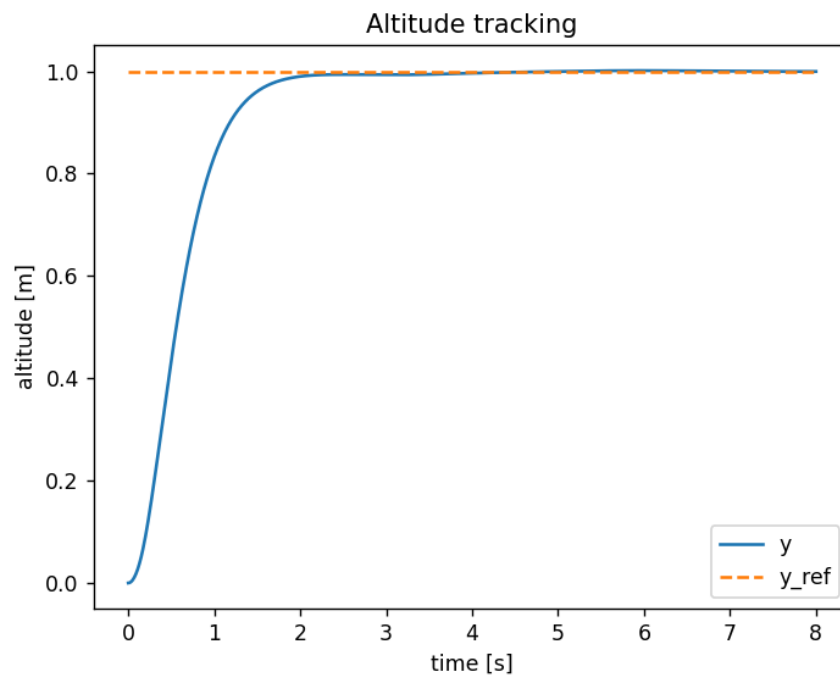
Στο σημείο αυτό να αναφέρουμε ότι είναι

$$\begin{bmatrix} v_x \\ v_y \end{bmatrix} = R(\theta)^T = \begin{bmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{bmatrix} \begin{bmatrix} \dot{x} \\ \dot{y} \end{bmatrix} (1.11)$$

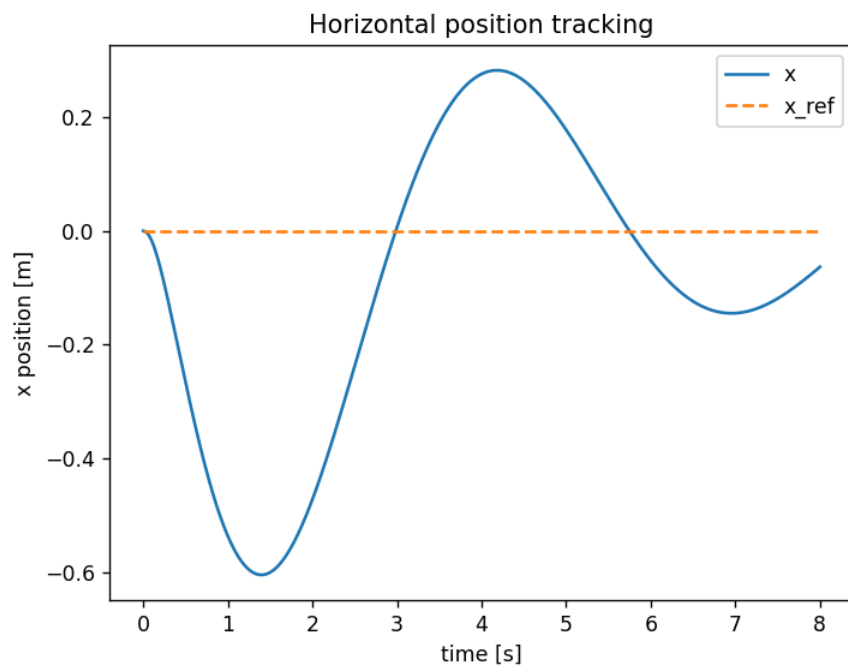
Τρέχοντας επομένως τον σχετικό κώδικα, λαμβάνουμε τα εξής αποτελέσματα (θεωρούμε αρχική κατάσταση $x_0 = [0 \ 0 \ 0.25 \ 0 \ 0 \ 0]^T$ (1.12).



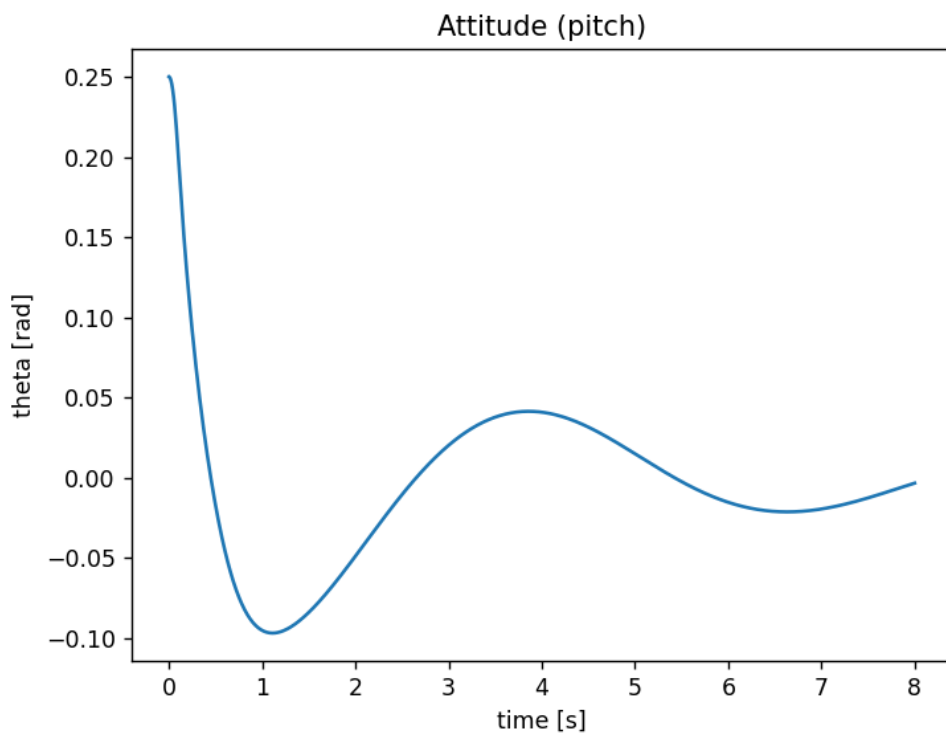
Εικ. 1.1: *x-y trajectory* που ακολουθεί το *quadrotor*



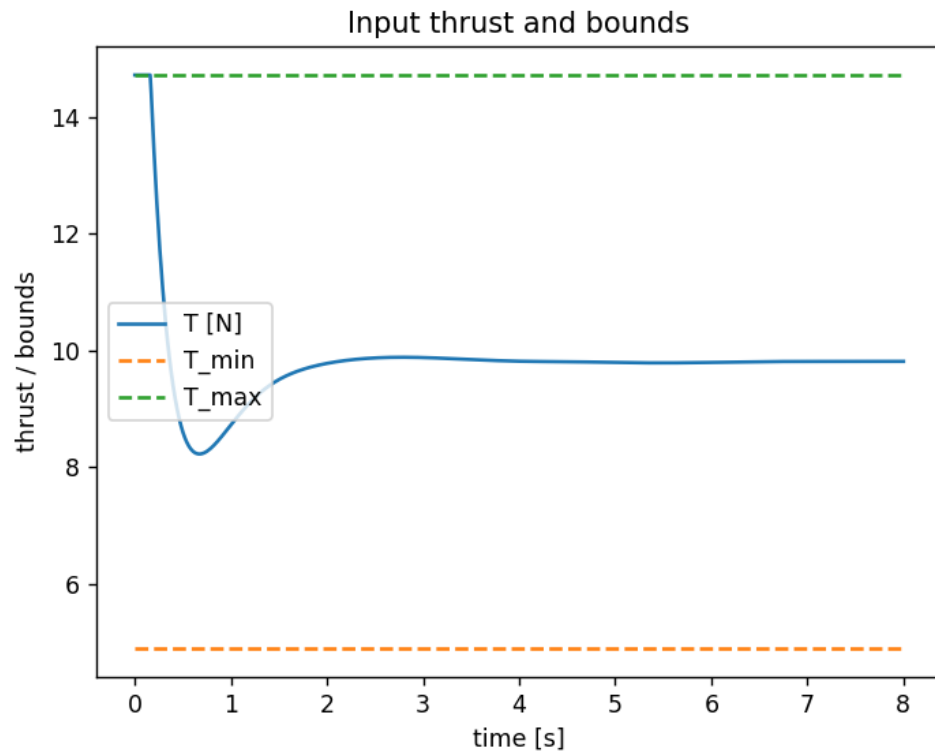
Εικ. 1.2 *Tracking* για το *y*



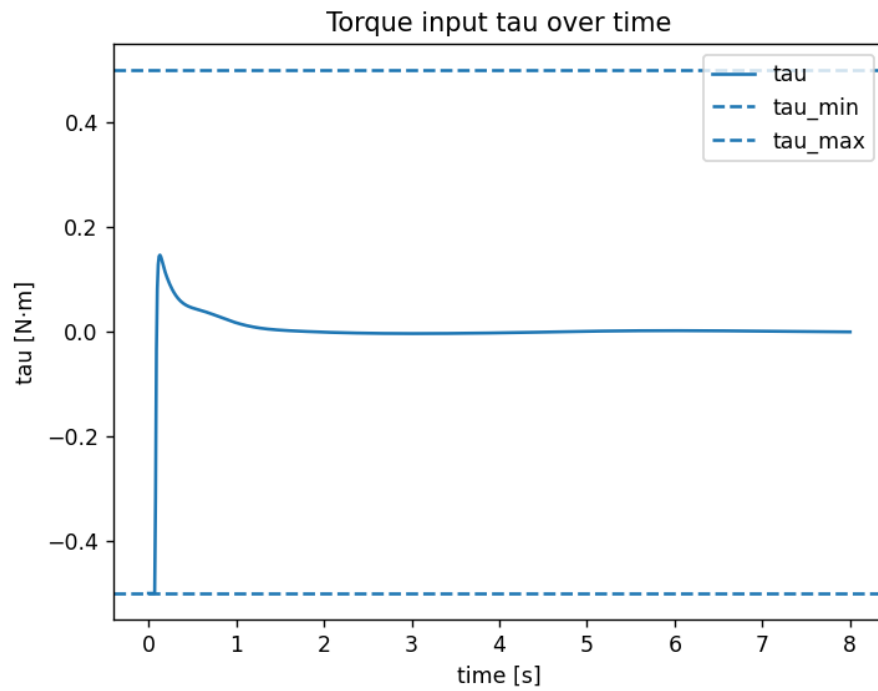
Εικ. 1.3: Tracking για το x



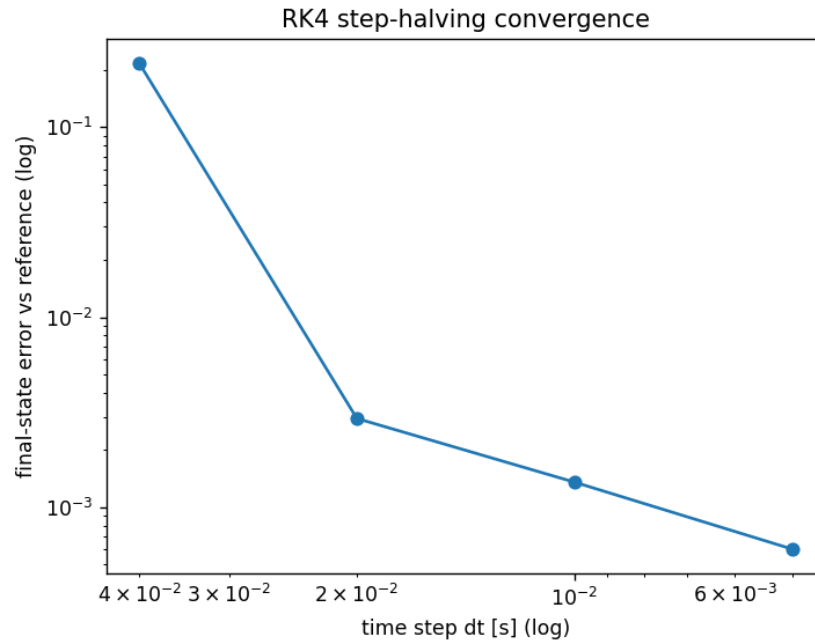
Εικ. 1.4: $\theta(t)$



Εικ. 1.5: $T(t)$ όπως προκύπτει από τον ελεγκτή



Εικ. 1.6: $\tau(t)$ όπως προκύπτει από τον ελεγκτή



Εικ. 1.7: Σύγκλιση της μεθόδου *RK4* αφού με μικρότερο dt μικραίνει το *error*

Άσκηση 2

Τώρα έχουμε να σχεδιάσουμε έναν ελεγκτή LQR για το σύστημά μας. Αρχικά, γραμμικοποιούμε το τελευταίο γύρω από το σημείο ισορροπίας x_{eq} . Πρέπει όμως να βρούμε το σημείο ισορροπίας:

Είναι $\dot{x} = 0$ (2.1). Μετά από σχετικές πράξεις καταλήγουμε στο συμπέρασμα:

$$x_{eq} = \begin{bmatrix} \alpha \\ \beta \\ 0 \\ 0 \\ 0 \end{bmatrix} - \alpha, \beta \in R \text{ (2.2), με εισόδους:}$$

$$u_{eq} = \begin{bmatrix} mg \\ 0 \end{bmatrix} \text{ (2.3)}$$

Επίσης:

$$\frac{\partial}{\partial \theta} \begin{bmatrix} v_x \\ v_y \end{bmatrix} = \begin{bmatrix} -\sin\theta & -\cos\theta \\ \cos\theta & -\sin\theta \end{bmatrix} \begin{bmatrix} \dot{x} \\ \dot{y} \end{bmatrix} \text{ (2.4)}$$

Κατά συνέπεια:

$$A = \frac{\partial f}{\partial x}|_{(x_{eq}, u_{eq})} = \begin{bmatrix} 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & -g & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & -\frac{k_\theta}{J} \end{bmatrix} \text{ (2.5)}$$

$$B = \frac{\partial f}{\partial u}|_{(x_{eq}, u_{eq})} = \begin{bmatrix} 0 & 0 \\ 0 & 0 \\ 0 & 0 \\ 0 & 0 \\ \frac{1}{m} & 0 \\ 0 & \frac{1}{J} \end{bmatrix} \text{ (2.6)}$$

Το *infinite horizon LQR* πρόβλημα κατά τα γνωστά έχει ως εξής:

$$J = \sum_{k=0}^{\infty} \left(\frac{1}{2} x_k^T Q x_k + \frac{1}{2} u_k^T R u_k \right)$$

$$\text{με } x_{k+1} = A x_k + B u_k \quad (2.7)$$

Με Q, R θετικά ημιορισμένο και θετικά ορισμένο αντιστοίχως. Ειδικότερα:

$$Q = \text{diag}(5, 5, 10, 1, 3, 1) \quad (2.8)$$

$$R = \text{diag}(0.02, 0.01) \quad (2.9)$$

Χρησιμοποιούμε *Riccati Recursion* για να το λύσουμε:

$$P_K = Q_K$$

$$K_k = (R_k + B_k^T P_{k+1} B_k)^{-1} B_k^T P_{k+1} A_k$$

$$P_k = Q_k + A_k^T P_{k+1} (A_k - B_k K_k)$$

Έχουμε *infinite-horizon LQR*, πράγμα το οποίο σημαίνει ότι το εφαρμόζουμε τις παραπάνω εξισώσεις μέχρι οι πίνακες P, K να συγκλίνουν σε σταθερές τιμές (οι πίνακες A, B είναι χρονικά αμετάβλητοι). Έχοντας λύσει το εν λόγω *LQR* πρόβλημα θα έχουμε έναν ελεγκτή της μορφής:

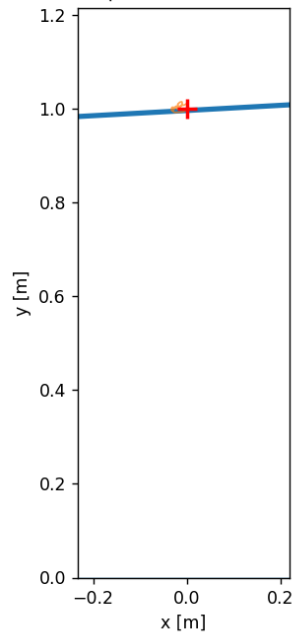
$$u = u_{eq} - K(x - x_{eq}), K \text{ είναι ο πίνακας που βρίσκουμε} \quad (2.10)$$

Επειδή το *LQR* πρόβλημα δεν επιδέχεται ανισοτικούς περιορισμούς, υπολογίζουμε το νόμο ελέγχου όπως προκύπτει από την προηγούμενη σχέση, και κάνουμε κατάλληλο *clamp* στο αποτέλεσμα μας.

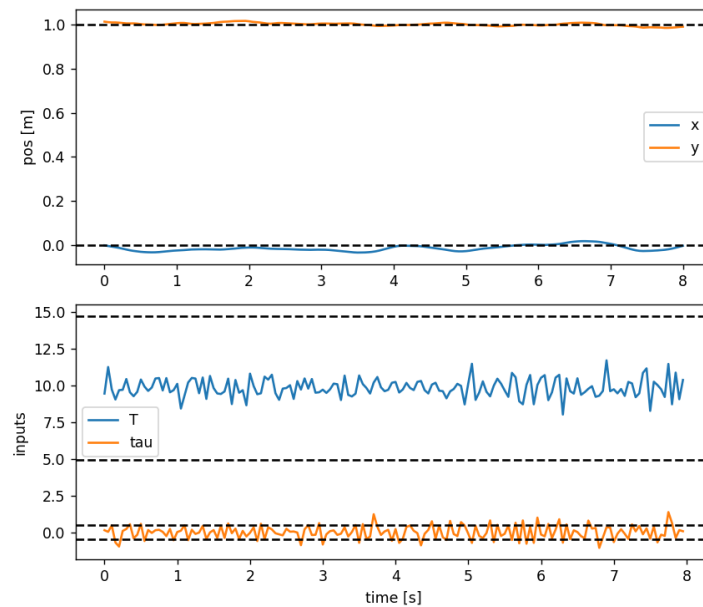
Τώρα για να εκτιμήσουμε πόσο μακριά από το σημείο γραμμικοποίησης ο *LQR* ελεγκτής μας μπορεί να ελέγξει το σύστημα, εμείς αρχικά επιλέγουμε μία κατεύθυνση να απομακρυνθεί το *quadrotor* και αποφασίζουμε εάν πράγματι έχουμε επιτυχία μετρώντας τα τελικά *errors*. Εάν έχουμε επιτυχία, τότε διπλασιάζουμε το πόσο μακριά ξεκινάμε από το σημείο ισορροπίας και επαναλαμβάνουμε την ίδια διαδικασία. Κάποια στιγμή φτάνουμε σε σημείο που έχουμε αποτυχία, οπότε για να κάνουμε μία εκτίμηση παίρνουμε το μέσο όρο της αντίστοιχης συντεταγμένης.

Τρέχοντας τον σχετικό κώδικα (βλ αρχείο *RS_II_HW2_Ex2*) λαμβάνουμε τα αποτελέσματα:

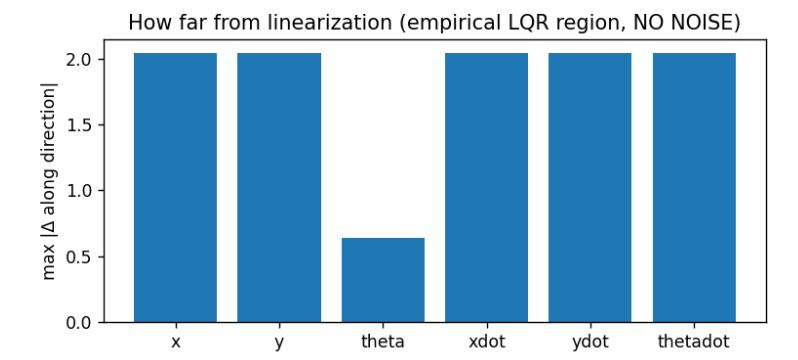
DLQR about hover (demo with mild noise) | $t=4.10s$



Εικ. 2.1: Προσομοίωση του συστήματος



Εικ. 2.2: Εξέλιξη των συντεταγμένων x,y σε σχέση με το reference (dashed γραμμή) –πάνω-και thrust/ροπή σε σχέση με τα επιτρεπτά όρια –κάτω-έχουμε προσθέσει θόρυβο.



Εικ. 2.3: Πόσο μακριά από το *linearization* δουλεύει αρκετά καλά ο *LQR controller*

Άσκηση 3

Τώρα έχουμε να σχεδιάσουμε για το σύστημά μας έναν MPC ελεγκτή. Το πρόβλημα έχει ως εξής:

$$\begin{aligned} \min_{\{x_i, u_i\}_{i=0}^{H-1}} \quad & \sum_{i=0}^{H-1} \frac{1}{2} ((x_i - x_i^{\text{ref}})^\top Q (x_i - x_i^{\text{ref}}) + (u_i - u_i^{\text{ref}})^\top R (u_i - u_i^{\text{ref}})) \\ & + \frac{1}{2} (x_H - x_H^{\text{ref}})^\top P (x_H - x_H^{\text{ref}}) \\ \text{s.t. } \quad & x_{i+1} = A_d x_i + B_d u_i, \quad i = 0..H-1 \\ & u_{\min} \leq u_i \leq u_{\max}. \end{aligned}$$

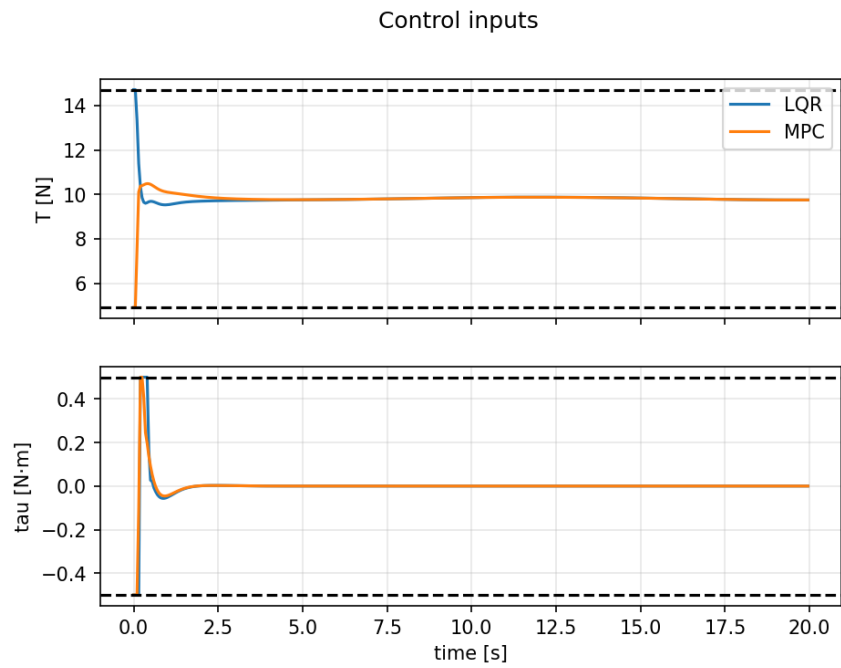
(λύνουμε σε κάθε βήμα ένα τέτοιο QP πρόβλημα)

Η διαφορά με τον *LQR controller* έγκειτα στο ότι πρώτον κοιτάζουμε για έναν πεπερασμένο ορίζοντα καταστάσεων του ελεγκτή (άρα έχουμε *penalty* για το τέλος-ο όρος έξω από το άθροισμα). Επίσης, υπάρχουν ενσωματωμένα τα *input constraints*, οπότε δεν απαιτείται *clamping* όπως στην περίπτωση του *LQR*. Εμείς σε κάθε βήμα, μετράμε την κατάσταση του συστήματος, βρίσκουμε *references* για τα επόμενα *H steps* και λύνουμε το QP πρόβλημα, οπότε βρίσκουμε μια ακολουθία $(u_0^*, u_1^*, \dots, u_{H-1}^*)$ και $(x_1^*, x_2^*, \dots, x_H^*)$. Εμείς εφαρμόζουμε μόνο το πρώτο *control*, οπότε πηγαίνουμε στην επόμενη κατάσταση (χρησιμοποιώντας το μη γραμμικό μας μοντέλο) και προχωράμε στο επόμενο χρονικό βήμα $t_k + dt$, οπότε και επαναλαμβάνουμε την ίδια διαδικασία. Οπότε το αναμενόμενο είναι να είναι πιο *robust*, αφού δεν υπολογίζει απλά έναν πίνακα αλλά online το βέλτιστο *input*. Επίσης, ειδικά στη συγκεκριμένη άσκηση θέλουμε το *quadrotor* να ακολουθήσει μια κυκλική τροχιά. Επομένως, έχουμε τα *reference trajectories*:

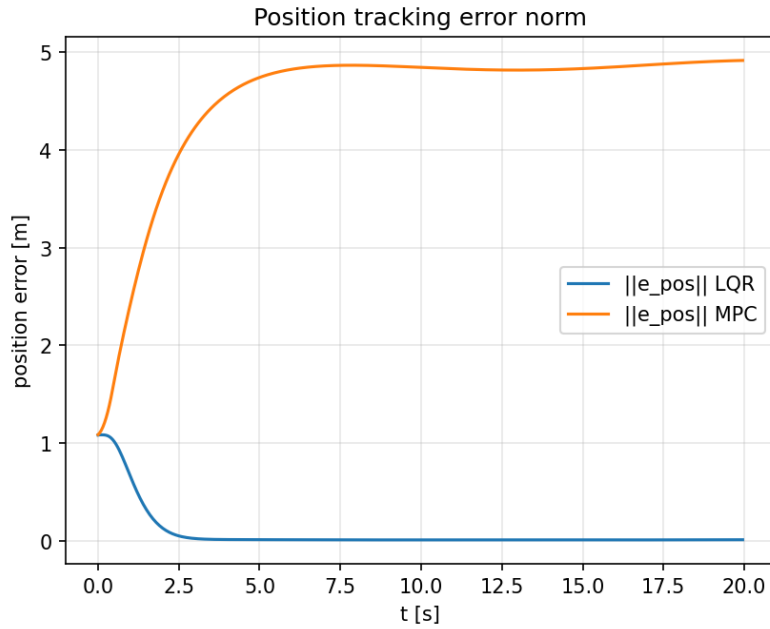
$$x_{ref}(t) = A + B \cos(\omega t) \quad (3.1)$$

$$y_{ref}(t) = C + D \sin(\omega t) \quad (3.2)$$

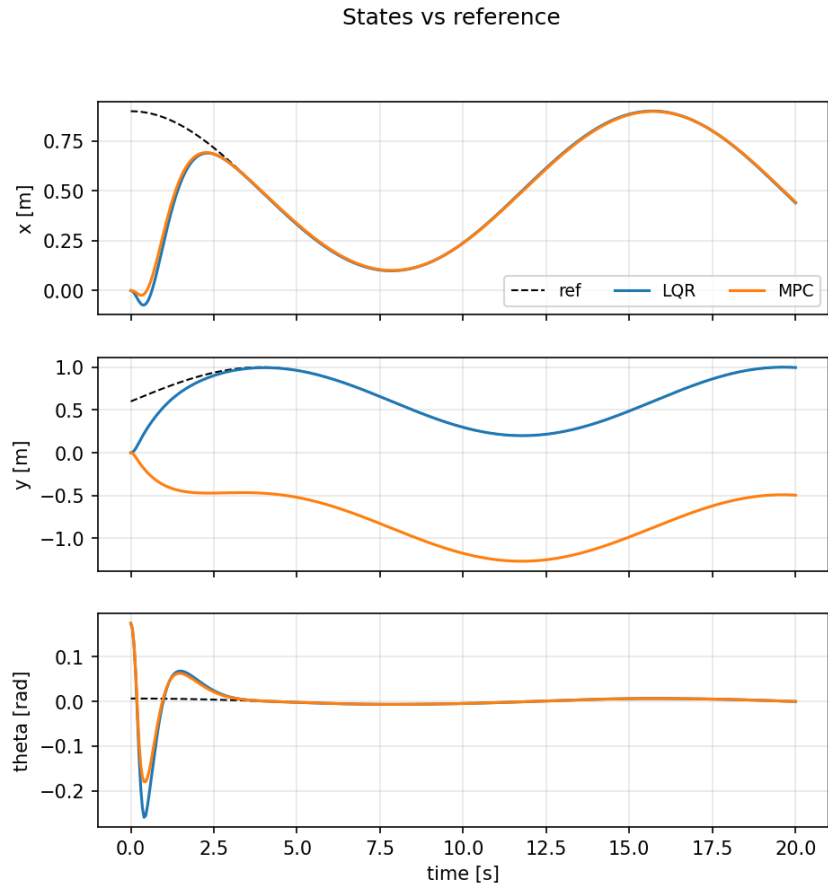
(οπότε βρίσκουμε άμεσα και τα $\dot{x}_{ref}, \dot{y}_{ref}$). Επίσης, θεωρώντας, $\dot{\theta}_{ref} = \ddot{\theta}_{ref} = 0$ (3.3), θα είναι και $\tau_{ref} = 0$ (3.4). Τώρα για το θ_{ref} , τα πράγματα είναι πιο περίπλοκα. Αρχικά, απαλοΐφουμε το T , που εμφανίζεται στις εξισώσεις των *continuous dynamics* στα $\dot{x}, \dot{y}$. Οπότε λαμβάνουμε μια εξίσωση ως προς θ , την οποία λύνουμε αριθμητικά. Εν συνεχεία, βρίσκουμε άμεσα το *thrust force* T μέσω μίας από τον δύο σχετικών εξισώσεων. Τρέχοντας επομένως τον σχετικό κώδικα (μέσω *docker* αυτή την φορά-βλ. αρχείο *RS_II_HW2_Ex3.py*), λαμβάνουμε τα εξής αποτελέσματα- $H = 24$ (3.4):



Εικ. 3.1: *LQR vs MPC control inputs*. Με εξαίρεση την αρχή, από ένα σημείο και μετά επί της ουσίας ταυτίζονται

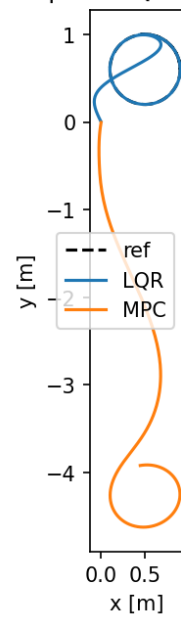


Εικόνα 3.2: *xy RMS error* για τους δύο ελεγκτές

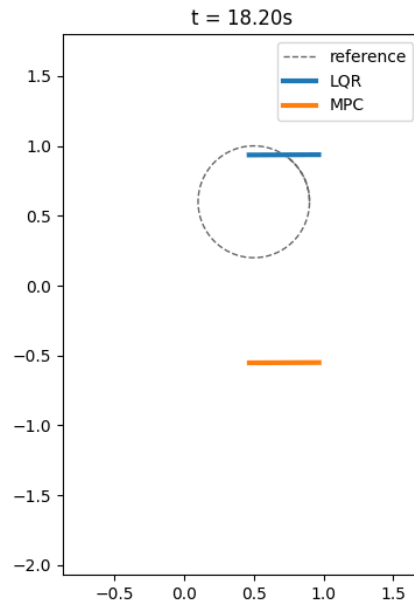


Εικόνα 3.3: Πώς εξελίσσονται τα x, y, θ με βάση τους δύο controllers. Παρατηρούμε ότι στο y έχουμε μια σημαντική διαφορά.

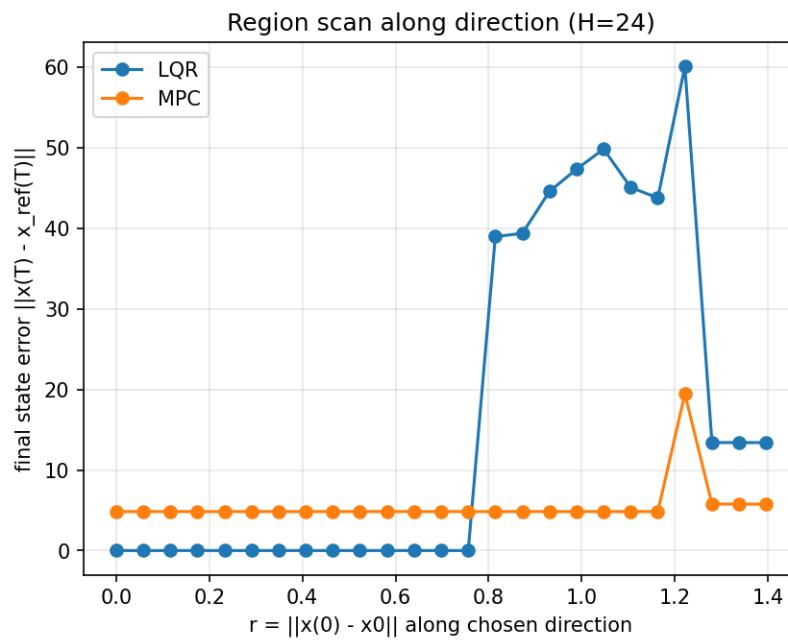
XY path: LQR vs MPC



Εικόνα 3.4: Κυκλική τροχιά για τους δύο ελεγκτές. Παρατηρούμε ότι ο MPC είναι πιο αργός.



Εικόνα 3.5 (βλ. αρχείο *quad_lqr_mpc.gif*) animation του συστήματος με βάση τους δύο ελεγκτές



Εικ. 3.6: Σχέση του τελικού error σε σχέση με το πόσο μακριά ξεκινάμε από το σημείο ισορροπίας κατά τη γωνία θ

Πάλι για τον *MPC controller*, προκειμένου να δούμε πόσο μπορούμε να απομακρυνθούμε από το σημείο γραμμικοποίησης, ακολουθούμε την ίδια ακριβώς διαδικασία με το προηγούμενο *task*, μόνο που τώρα $r = 0$ (3.5), λόγω του πολύ μικρού περιθώριου λάθους 0.5. Έγινε προσπάθεια με μεγαλύτερα *tolerances* αλλά ο κώδικας δεν εκτελείται/παίρνει αρκετή ώρα προκειμένου να έχουμε αποτελέσματα. Επίσης στο γεγονός αυτό συμβάλλει και το ότι με τον *MPC controller*, το σύστημά μας μετατοπίζεται περισσότερο προς τα κάτω, οπότε έχουμε ένα μεγάλο *error* στην κατεύθυνση y , πράγμα που ωθεί το r (το πόσο μπορούμε να απομακρυνθούμε δηλαδή από την αρχική κατάσταση) στην τιμή 0.-βλ. επίσης εικόνα 3.6 πότε δεν είναι δυνατόν να ξεκινήσουμε από οποιοδήποτε αρχικό *point* και *orientation* και να έχουμε σύγκλιση στο σημείο ισορροπίας. Τέλος αφού το σημείο ισορροπίας είναι το (2.2), μπορούμε να επιλέξουμε όποιες τιμές θέλουμε για τα x και y .

Σημείωση για τον κώδικα: Λόγω προβλημάτων εγκατάστασης της βιβλιοθήκης *proxsuite*, τρέχουμε σε *docker* το σχετικό αρχείο με τις εξής εντολές:

```
docker build -t rs_hw2_ex3
```

```
docker run --rm -v "${PWD}:/app" rs_hw2_ex3
```

A-ΠΑΡΑΡΤΗΜΑ- Derivation των εξισώσεων του συστήματος

Τα 3 πρώτα στοιχεία του $\dot{x}$ είναι προφανή.

Για τα υπόλοιπα 3 στοιχεία:

Η συνολική δύναμη (στο *body frame*) είναι:

$$F_{b,tot} = \begin{bmatrix} T + f_x \\ f_y - mg \end{bmatrix} \quad (A.1)$$

Οπότε, στο world frame θα είναι:

$$F_{w,tot} = \begin{bmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{bmatrix} F_{b,tot} = \begin{bmatrix} \cos\theta f_x - \sin\theta(T + f_y) \\ \sin\theta f_x + \cos\theta(T + f_y) \end{bmatrix} \quad (A.2)$$

Άρα από το νόμο του Νεύτωνα για τη μεταφορική κίνηση:

$$\Sigma F = ma \quad (A.3) ,$$

θα είναι $\ddot{x} = \frac{1}{m} \cos\theta f_x - \sin\theta(T + f_y)$ (A.4) και

$$\ddot{y} = \frac{1}{m} \sin\theta f_x + \cos\theta(T + f_y) - g \quad (A.5)$$

Τώρα, υπολογίζουμε τη συνολική ροπή:

$$\tau_{tot} = \tau + f_\theta = \tau - k_\theta \dot{\theta} \quad (A.6)$$

Επομένως από το νόμο του Νεύτωνα για την περιστροφική κίνηση:

$$\Sigma \tau = J \ddot{\theta} \quad (A.7)$$

$$\Delta\eta\lambda\alpha\delta\acute{\eta} \ddot{\theta} = \frac{\tau - k_\theta}{J} \quad (A.8)$$

Οπότε από τις (A.5), (A.7), (A.8) έχουμε κάνει *derive* τα *dynamics* του συστήματός μας.